

# Signing a document about signing

This file quotes the MAdES opening marker in full, more than once, and contains a complete signature block as an example. It is also itself signed with MAdES.

Until specification v1.4, those two things could not both be true.

## What used to happen

A verifier finds a signature by scanning for the marker `<!-- makes-sig`. The sentence you have just read was enough to break a document: the scan could not tell a signature from a description of one, so it found a block here, in prose, with no signature inside it. The reader was then told that a perfectly sound document could not be checked.

That is a false failure, and false failures cost precisely what false successes cost. Worse, this one landed on exactly the documents most likely to be read by someone deciding whether to trust the format at all: the tutorial, the issue, the specification itself.

The earlier example beside this one, `05-a-real-signed-document.md`, contains a paragraph explaining that it never writes the marker in full — because doing so would break its own signature. That paragraph was true when it was signed.

It is still there, and it stays there. **A signed document is not edited to make it say something more flattering.** That is the entire property this format exists to provide, and it would be a strange thing to abandon in the example demonstrating it. The document is superseded, not corrected.

## The two rules

Specification v1.4, §a.1:

1. The opening marker counts only at the **start of a line**.
2. It is never recognised **inside a fenced code block**.

So a sentence may name `<!-- makes-sig` inline, as this one does, without consequence. And a fenced example may show a whole block:

```
<!-- makes-sig
version: 5
algorithm: ecdsa-p256
signer: someone@example.com
signer-kind: human
commitment: approval
signed-at: 2026-01-01T00:00:00Z
certificate-chain:
  - MIIB...
signature: 7a3f...8b2c
-->
```

That is an illustration. The real one is at the bottom of this file: at column zero, outside any fence, hidden by your viewer and visible in your editor.

## Why the rules are shaped this way

Both are byte-level. Neither needs a Markdown parser, and that is the point rather than an optimisation.

An implementation that has to *understand* a document in order to find its signature is an implementation that will eventually disagree with the next one about where the signature was. Disagreement about the location of a signature does not read to a person as a parsing difference. It reads as tampering.

So the fence rule follows CommonMark exactly — three or more backticks or tildes, indented by at most three spaces, closed by at least as many of the same character and nothing else on the line. Both halves earn their place. Close on any marker at all, and a short fence ends a longer one early, exposing the example inside it. Ignore the indentation allowance, and a fence indented by two spaces stays invisible to the scanner while still rendering as code to the reader. Either way the hole reopens in a corner.

The marker itself is held to the stricter rule: column zero, no indentation. An indented marker is an indented code block, and renders as code too.

The result is that a verifier skips exactly what a reader sees as code. That correspondence is the whole design.

## What this document is not

It is **not a revision of 05 in the sense of §a.7**. That mechanism — the `revision` and `supersedes` fields — chains a signature to the digest of what an earlier signature covered, and it requires the same signer, a verifying earlier block, and a chain to a trusted anchor. This file is a separate document carrying its own first signature.

The relationship between the two is editorial. It is stated here, in prose that falls under the signature, rather than asserted in a field that means something narrower and stronger than what is true.

## Checking it

The procedure has not changed, and is set out in 05: rebuild the signing input from the file, verify against the certificate the file carries, check that certificate against an anchor you already hold, and check that the timestamp falls inside the certificate's validity window.

An expired certificate here is normal and is not a finding. The certificate lived for minutes; the timestamp is what carries the signature past it.

The one thing worth adding: if your verifier reports two blocks in this file, it has not implemented §a.1 as of v1.4, and it is that verifier reporting on itself.